

StockDex — ALM

Analyse rétrospective

Bilan critique individuel — StockDex

Jeremy Lardet © 2026

1. Ce qui a bien fonctionné

Le choix technologique SQLite + bettersqlite3

La contrainte «pas de BDD complexe» m'a poussé vers SQLite. Ce choix s'est avéré excellent : l'API synchrone de bettersqlite3 a éliminé toute la complexité des callbacks ou des promesses chaînées pour les requêtes. Les transactions ACID ont permis de gérer l'atomicité du trade sans bibliothèque externe. Le file-based storage a simplifié le déploiement sur Render (un disque attaché suffit). En termes ALM, ce choix a accéléré la phase de développement sans sacrifier la qualité.

GitHub Actions comme filet de sécurité

Configurer le pipeline CI dès le premier jour (et non à la fin) a été une excellente décision. Le pipeline a détecté deux régressions avant qu'elles n'atteignent main : une erreur de validation sur le trade et un bug de calcul de poids dans drawCards. Sans CI, ces bugs auraient été découverts en démo. L'effort de configuration (< 2h) a été largement rentabilisé.

La séparation Vercel / Render

Héberger les assets statiques React sur Vercel CDN et l'API sur Render a eu un impact mesurable sur la latence perçue. Les assets sont servis depuis le PoP le plus proche, tandis que Render gère uniquement les requêtes API. Cette architecture a également facilité le débogage : les problèmes frontend et backend sont isolés dans des logs distincts.

2. Ce qui a moins bien fonctionné

L'absence de tests en amont du code

J'ai écrit les tests après le code pour la majorité des modules, ce qui est contraire au TDD. Résultat : plusieurs fonctions ont dû être refactorisées pour être testables (injection de dépendances manquante, effets de bord sur la BDD). Le coût de rétrofit des tests est plus élevé que de les écrire au départ — c'est exactement ce que le cycle Red-Green-Refactor cherche à éviter.

La race condition sur le trade

En conception, j'avais identifié le besoin de transaction SQLite mais n'avais pas formalisé le cas de la requête concurrente (deux joueurs acceptant simultanément le même trade). Ce scénario n'apparaissait dans aucun cas de test. La résolution en développement a pris environ 3 heures qui auraient pu être économisées avec un diagramme de séquence plus détaillé couvrant les flux d'exception.

La dette du state management React

Choisir des useState flat propagés par props était raisonnable pour un prototype. Cependant, au-delà de 5 composants, le prop drilling est devenu pénible à maintenir. Ce compromis (rapidité initiale vs maintenabilité) aurait dû être documenté dans un ADR dès le sprint 1 pour éviter la surprise en sprint 2.

3. Ce que je ferais différemment

Écrire les ADR dès les premières décisions

Architecture Decision Records (cf. Module 3) : j'aurais documenté les décisions clés (SQLite vs PostgreSQL, props vs Context, GitHub Flow vs GitFlow) dès qu'elles ont été prises. Cela aurait permis d'expliquer rapidement les choix aux pairs et d'éviter les discussions répétées sur des décisions déjà closes.

Adopter TDD pour les composants critiques

Pour le moteur de fluctuation et le service de trade — les deux composants avec le plus de logique métier — j'aurais appliqué le cycle Red-Green-Refactor dès le départ. Les tests auraient servi de documentation vivante et forcé un design plus modulaire.

Déployer dès le sprint 1

J'ai déployé l'application à la fin du sprint 2. Déployer dès la fin du sprint 1 (même une version minimale) aurait révélé les problèmes d'environnement (cold start Render, CORS, variables d'env) bien plus tôt. La règle empirique «déployez tôt, déployez souvent» du TP est tout à fait justifiée — je l'ai appris à mes dépens.